QACM LAB 1–9

CODE-ONLY PRACTICAL REVISION SHEET

Nine experiments • Cellwise copy-paste format • A4 print layout

Code is organized into separate notebook cells. Run cells in order in Google Colab.

Source-faithful note: Experiment 8 is labelled as a QFT/QPE-style demonstration because the supplied circuit is not a complete Shor factoring implementation.

EXPERIMENT 1

Single-Qubit Quantum Gates — X, Z, and Hadamard

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS AND CIRCUIT

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

qc = QuantumCircuit(1)
```

CELL 3 — APPLY X, Z, H GATES

```
qc.x(0)
qc.z(0)
qc.h(0)
```

CELL 4 — DRAW CIRCUIT

```
print(qc.draw())
qc.draw("mpl")
```

CELL 5 — STATEVECTOR AND BLOCH SPHERE

```
state = Statevector.from_instruction(qc)
print("Final state:", state)
plot_bloch_multivector(state)
```

EXPERIMENT 2

Simulation and Visualisation of a 2-Qubit Entangled State

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.visualization import (
    plot_histogram, plot_state_city, plot_state_hinton,
    plot_bloch_multivector
)
```

CELL 3 — BUILD ENTANGLED STATE

```
qc = QuantumCircuit(2, 2)
qc.x(1)
qc.h(0)
qc.cx(0, 1)
```

CELL 4 — MEASURE AND DRAW

```
qc.measure([0, 1], [0, 1])
print(qc.draw())
qc.draw("mpl")
```

CELL 5 — SIMULATE AND PLOT RESULTS

```
simulator = AerSimulator()
result = simulator.run(transpile(qc, simulator), shots=1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

CELL 6 — STATE VISUALISATIONS (UNMEASURED CIRCUIT)

```
state_qc = QuantumCircuit(2)
state_qc.x(1)
state_qc.h(0)
state_qc.cx(0, 1)
state = Statevector.from_instruction(state_qc)
plot_state_city(state)
plot_state_hinton(state)
plot_bloch_multivector(state)
```

EXPERIMENT 3

Build a Circuit to Generate a Bell State

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.visualization import (
    plot_histogram, plot_state_city, plot_state_hinton, plot_state_qsphere
)
```

CELL 3 — CREATE BELL CIRCUIT

```
bell = QuantumCircuit(2)
bell.h(0)
bell.cx(0, 1)
print(bell.draw())
bell.draw("mpl")
```

CELL 4 — STATE VISUALISATIONS

```
state = Statevector.from_instruction(bell)
print(state)
plot_state_city(state)
plot_state_hinton(state)
plot_state_qsphere(state)
```

CELL 5 — MEASUREMENT SIMULATION

```
measured = QuantumCircuit(2, 2)
measured.h(0)
measured.cx(0, 1)
measured.measure([0, 1], [0, 1])
simulator = AerSimulator()
result = simulator.run(transpile(measured, simulator), shots=1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

EXPERIMENT 4

Superdense Coding using Qiskit

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
```

CELL 3 — SUPERDENSE CODING FUNCTION

```
def superdense_encode(message):
    if message not in {"00", "01", "10", "11"}:
        raise ValueError("Message must be 00, 01, 10, or 11")

    qc = QuantumCircuit(2, 2)
    qc.h(0)
    qc.cx(0, 1)

    if message == "01":
        qc.x(0)
    elif message == "10":
        qc.z(0)
    elif message == "11":
        qc.x(0)
        qc.z(0)

    qc.cx(0, 1)
    qc.h(0)
    qc.measure([0, 1], [0, 1])
    return qc
```

CELL 4 — RUN ALL FOUR MESSAGES

```
simulator = AerSimulator()
for message in ["00", "01", "10", "11"]:
    qc = superdense_encode(message)
    result = simulator.run(transpile(qc, simulator), shots=1024).result()
    counts = result.get_counts()
    print("Sent:", message, "Received counts:", counts)
    plot_histogram(counts, title=f"Message {message}")
```

EXPERIMENT 5

Implement a Multi-Qubit SWAP Using CNOT

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
```

CELL 3 — SWAP USING THREE CNOTS

```
def swap_using_cnot(qc, a, b):
    qc.cx(a, b)
    qc.cx(b, a)
    qc.cx(a, b)

qc = QuantumCircuit(2)
qc.x(0)
swap_using_cnot(qc, 0, 1)
print(qc.draw())
print(Statevector.from_instruction(qc))
```

CELL 4 — VERIFY ALL BASIS STATES

```
for value in range(4):
    qc = QuantumCircuit(2)
    if value & 1:
        qc.x(0)
    if value & 2:
        qc.x(1)
    swap_using_cnot(qc, 0, 1)
    print(format(value, "02b"), "->", Statevector.from_instruction(qc))
```

CELL 5 — THREE-QUBIT CHAIN

```
qc3 = QuantumCircuit(3)
qc3.x(0)
swap_using_cnot(qc3, 0, 1)
swap_using_cnot(qc3, 1, 2)
print(qc3.draw())
print(Statevector.from_instruction(qc3))
```

EXPERIMENT 6

Deutsch–Jozsa Algorithm — Balanced and Constant Functions

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
```

CELL 3 — DEFINE ORACLE

```
def dj_oracle(case, n):
    oracle = QuantumCircuit(n + 1)
    if case == "constant_one":
        oracle.x(n)
    elif case == "balanced":
        for q in range(n):
            oracle.cx(q, n)
    elif case != "constant_zero":
        raise ValueError("Use constant_zero, constant_one, or balanced")
    return oracle
```

CELL 4 — BUILD DEUTSCH–JOZSA CIRCUIT

```
def deutsch_jozsa(case, n=3):
    qc = QuantumCircuit(n + 1, n)
    qc.x(n)
    qc.h(n)
    qc.h(range(n))
    qc.compose(dj_oracle(case, n), inplace=True)
    qc.h(range(n))
    qc.measure(range(n), range(n))
    return qc
```

CELL 5 — RUN SIMULATION

```
simulator = AerSimulator()
for case in ["constant_zero", "constant_one", "balanced"]:
    qc = deutsch_jozsa(case, 3)
    result = simulator.run(transpile(qc, simulator), shots=1024).result()
    counts = result.get_counts()
    is_constant = set(counts) == {"000"}
    print(case, counts)
    print("Classification:", "constant" if is_constant else "balanced")
    print(qc.draw())
```

EXPERIMENT 7

Grover’s Algorithm to Search for a Marked State

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
```

CELL 3 — ORACLE AND DIFFUSION FOR |11>

```
qc = QuantumCircuit(2, 2)
qc.h([0, 1])

# Phase oracle marks |11>
qc.cz(0, 1)

# Diffusion operator
qc.h([0, 1])
qc.x([0, 1])
qc.h(1)
qc.cx(0, 1)
qc.h(1)
qc.x([0, 1])
qc.h([0, 1])
```

CELL 4 — MEASURE AND RUN

```
qc.measure([0, 1], [0, 1])
print(qc.draw())
simulator = AerSimulator()
result = simulator.run(transpile(qc, simulator), shots=2048).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

EXPERIMENT 8

Quantum Fourier Transform and QPE-Style Demonstration (N = 15)

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.circuit.library import QFT
from qiskit.visualization import plot_histogram
```

CELL 3 — QFT AND INVERSE QFT

```
n = 3
qft = QFT(n, do_swaps=True)
iqft = QFT(n, inverse=True, do_swaps=True)
print("QFT circuit:")
print(qft.decompose().draw())
print("Inverse QFT circuit:")
print(iqft.decompose().draw())
```

CELL 4 — QPE-STYLE DEMONSTRATION CIRCUIT

```
qc = QuantumCircuit(4, 3)
qc.x(3)
qc.h([0, 1, 2])

# Controlled phase rotations for a QPE-style demonstration
qc.cp(3.141592653589793 / 2, 0, 3)
qc.cp(3.141592653589793 / 2, 1, 3)
qc.cp(3.141592653589793 / 2, 2, 3)
qc.append(iqft.to_gate(), [0, 1, 2])
qc.measure([0, 1, 2], [0, 1, 2])
print(qc.draw())
```

CELL 5 — SIMULATE CIRCUIT

```
simulator = AerSimulator()
result = simulator.run(transpile(qc, simulator), shots=1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)

# Classical factor confirmation for N = 15
N = 15
print("Prime factors of", N, "are 3 and 5")
```

EXPERIMENT 9

Quantum Random Bitstring and Integer Generation

CELL 1 — INSTALLATION

```
!pip install qiskit qiskit-aer pylatexenc
```

CELL 2 — IMPORTS

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
```

CELL 3 — GENERATE A RANDOM 4-BIT STRING

```
n = 4
qc = QuantumCircuit(n, n)
qc.h(range(n))
qc.measure(range(n), range(n))
print(qc.draw())
```

CELL 4 — SIMULATE AND EXTRACT RANDOM VALUE

```
simulator = AerSimulator()
result = simulator.run(transpile(qc, simulator), shots=1).result()
counts = result.get_counts()
random_bits = next(iter(counts))
print("Counts:", counts)
print("Random bitstring:", random_bits)
print("Random integer:", int(random_bits, 2))
```